

Writeup: Crackme creed_1

Informations sur le Crackme

- **Équipe cible** : Non spécifiée
- **Nom du fichier** : writeup_02__creed1OK.txt
- **Difficulté estimée** : Non spécifiée
- **Flag découvert** : `M4ST3R_H4CK3R_42`

Résumé

Analyse initiale

J'ai commencé par examiner le fichier avec la commande `file` pour identifier son type:

```
file creed_1
```

Le résultat a montré qu'il s'agissait d'un exécutable ELF 64-bit pour Linux, statiquement lié et non strippé, ce qui signifie que les symboles de débogage étaient toujours présents pour faciliter l'analyse.

Outils Utilisés

Extraction du mot de passe

Pour trouver le mot de passe, j'ai examiné le contenu à l'adresse `0x4021a8` en utilisant GDB:

```
creed_1
```

```
(gdb) x/16bx 0x4021a8
```

Analyse Statique

Exploration des symboles

J'ai utilisé `nm` pour examiner les symboles du binaire:

```
nm creed_1
```

Cette commande a révélé de nombreux symboles avec des noms cryptiques comme `_r0`, `_r1`, `_l1`, `_e1`, etc., indiquant des fonctions et variables différentes dans le programme.

Désassemblage du programme

J'ai désassemblé le binaire pour comprendre son fonctionnement:

```
objdump -d creed_1 > creed_1_disasm.txt
```

En analysant le code, j'ai identifié plusieurs points clés:

Analyse Dynamique

Le programme contient également des mécanismes de protection anti-débogage:

Mesure du temps d'exécution à l'aide de rdtsc

Diverses opérations de mélange et de chiffrement pour rendre l'analyse statique plus difficile

Identification du Mécanisme de Validation

Le programme commence à _start qui appelle directement _r1

À l'adresse 0x4010c0, il vérifie que l'entrée a exactement 16 caractères (cmp \$0x10,%rcx)

La fonction _f_loop à l'adresse 0x401173 copie 16 octets depuis l'adresse 0x4021a8 vers 0x402240

L'entrée utilisateur est comparée avec ce contenu

Vérification

Pour confirmer que ce mot de passe est correct, j'ai exécuté le programme.

J'ai entré M4ST3R_H4CK3R_42 et le programme a affiché un message de succès.

Découverte du Flag

Le résultat a montré qu'il s'agissait d'un exécutable ELF 64-bit pour Linux, statiquement lié et non strippé, ce qui signifie que les symboles de débogage étaient toujours présents pour faciliter l'analyse.

Conclusion

Conclusion

Le crackme "creed_1" stockait le mot de passe à l'adresse 0x4021a8, qui est copié dans un buffer de comparaison lors de l'exécution. L'analyse du désassemblage et l'utilisation de GDB ont permis d'extraire directement la valeur.

Le flag est M4ST3R_H4CK3R_42.

Writeup

Writeup: Crackme creed_1

Ce qui a affiché

Ce qui a affiché:

```
0x4021a8: 0x4d 0x34 0x53 0x54 0x33 0x52 0x5f 0x48
0x4021b0: 0x34 0x43 0x4b 0x33 0x52 0x5f 0x34 0x32
```

J'ai converti ces valeurs hexadécimales en caractères ASCII:

```
0x4d = M   0x34 = 4   0x53 = S   0x54 = T   0x33 = 3   0x52 = R   0x5f = _
0x48 = H
```

0x34 = 4 0x43 = C 0x4b = K 0x33 = 3 0x52 = R 0x5f = _ 0x34 = 4
0x32 = 2

Ce qui donne

Ce qui donne: M4ST3R_H4CK3R_42

Particularités techniques

Particularités techniques :